Національний університет «Львівська політехніка»
Кафедра систем штучного інтелекту

ЛАБОРАТОРНА РОБОТА № 5

Географічні дані OpenStreetMap: від тегів до маршруту

Методичні вказівки до виконання

Інтелектуальний аналіз та візуалізація даних

4 години аудиторних · 4 бали

Львів · 2026

1. Про що ця робота

Ви — аналітик у туристичному застосунку. Замовник просить для заданого міста відібрати **десять найвартісніших історико-культурних об'єктів** і скласти піший маршрут: почати на залізничному вокзалі, обійти всі десять і повернутися на вокзал.

Звучить як одна задача. Насправді їх чотири, і кожна має свою пастку.

Крок	Питання, на яке він відповідає	Де пастка
Відбір	які об'єкти взагалі є пам'ятками?	у OSM немає тега «пам'ятка»
Рейтинг	які з них найвартісніші?	у даних немає стовпця «цінність»
Віддалі	скільки між ними йти?	дорога не є прямою лінією
Маршрут	у якому порядку обходити?	точний розв'язок можливий не завжди

Остання пастка — не технічна дрібниця. На десяти точках найкоротший маршрут обчислюється **точно**, за соті частки секунди. На тридцяти — не обчислюється взагалі, і причина не в терпінні. Саме з цього місця почнеться **лабораторна робота № 8**, де ту саму задачу розв'язуватимуть генетичним та мурашиним алгоритмами.

> Головна думка роботи, яку варто сформулювати для себе ще до початку:
>
> слово «найкращі» не є властивістю даних. Воно є властивістю критерію, який обирає аналітик — і за який він відповідає.

1.1. Що потрібно здати

Файл	Що містить
`ЛР5_прізвище.ipynb`	ноутбук із виконаними комірками; **усі виводи збережені**
`lr5_результати.json`	результати для лабораторної роботи № 8
`lr5_маршрут.html`	інтерактивна карта маршруту
звіт	останні комірки ноутбука: сім відповідей і декларація про ШІ

> Ноутбук із порожніми виводами до перевірки не приймається. Перед здачею виконайте «Restart & Run All» і переконайтесь, що всі комірки відпрацювали без помилок, а графіки й таблиці видно.

1.2. Як оцінюється

Критерій	0	2	3	4
Відбір об'єктів	теги взято навмання	фільтр працює	фільтр обґрунтовано	показано, що відкинуто і чому
Критерій рейтингу	немає	є формула	ваги пояснено	порівняно з альтернативним критерієм
Маршрут	не будується	будується	перевірено коректність	різницю методів пояснено числом
Карта	немає	точки на карті	маршрут по ребрах графа	видно різницю з прямими відрізками
Звіт	немає	формально	з числами	з висновком, якого не було в завданні

Остаточна оцінка — **мінімум** по рядках, а не середнє. Робота без карти не може бути оцінена на 4 бали, навіть якщо решта бездоганна.

1.3. Що треба знати до початку

- Лекція 6 — модель даних OSM, системи координат, просторове поєднання, індексація H3.
- Демонстраційний ноутбук лекції 6, особливо розділ 4.6 (два механізми завантаження) і акт 5 (маршрути).
- `pandas` на рівні лабораторних робіт № 1 і № 3.

2. Теорія, якої вистачить

2.1. Як влаштований OpenStreetMap

OSM зберігає три типи об'єктів і більше нічого:

Тип	Що це	Приклад
вузол (node)	точка з координатами	пам'ятник, вхід, лавка
шлях (way)	впорядкований список вузлів	вулиця, контур будівлі
відношення (relation)	група об'єктів із ролями	межа міста, маршрут автобуса

Сенс об'єкта задають **теги** — пари «ключ = значення». Тег `amenity=pharmacy` робить будівлю аптекою; без тегів будівля — просто контур.

> Один і той самий вид об'єкта трапляється в різних типах геометрії. Музей у власній будівлі — це way (полігон). Музей в орендованому приміщенні — node (точка). Якщо ваш фільтр лишає тільки полігони, ви мовчки втратили частину музеїв, усі пам'ятники та всі оглядові точки.

Де шукати пам'ятки

Тега «визначна пам'ятка» в OSM немає. Є два сімейства, які доводиться об'єднувати вручну:

Ключ	Характерні значення	Що означає
`historic`	`castle`, `church`, `monument`, `memorial`, `ruins`, `tower`, `yes`	об'єкт має історичну природу
`tourism`	`museum`, `gallery`, `attraction`, `artwork`, `viewpoint`	об'єкт показують туристам

Перетин цих сімейств неповний: сучасна галерея має лише `tourism`, руїни замку — лише `historic`. Тому беремо **об'єднання** умов, а не перетин. Повний перелік ключів — у довіднику Map Features, розділи **Historic** і **Tourism**.

2.2. Ліцензія: що з цими даними можна

Дані OSM поширюються за ліцензією **ODbL** (Open Database License). Вона вимагає двох речей: вказати авторство — «© OpenStreetMap contributors» — і поширювати похідні бази **під тією самою ліцензією**. Докладно: openstreetmap.org/copyright.

Для звіту це означає рядок із зазначенням джерела. Для реального застосунку — юридичне зобов'язання, яке варто прочитати до релізу, а не після.

2.3. Два способи дістати дані

Робоча програма вимагає порівняти обидва **за обсягом переданих даних і за часом**. Це не формальність: вибір між ними — проєктне рішення.

	Живий запит (Overpass / osmnx)	Знімок (GeoParquet)
свіжість	хвилини	станом на дату зрізу
швидкість	секунди–хвилини	миттєво
мережа	кожен запуск платить трафіком	нуль після першого разу
обмеження	сервіс обмежує частоту	файл треба зберігати, він застаріває
коли брати	малий район, потрібні свіжі дані	повторюваний аналіз, навчання

> Overpass — безкоштовний сервіс, і частоту він обмежує навмисно. Якщо тридцять студентів одночасно запитають своє місто, частина отримає відмову. Тому ноутбук побудовано так, щоб

живий запит виконувався ОДИН раз, а далі читався збережений знімок. Якщо запит не проходить — візьміть резервний знімок у викладача і зазначте це у звіті.

Якщо Overpass не відповідає

Це трапляється регулярно, і вікі OSM говорить про головний сервер прямо: він перевантажений, і високої надійності очікувати не варто. Симптоми бувають різні, і розрізняти їх корисно:

Що бачите	Що це означає	Що робити
HTTP 406	ви не представились: сервер не приймає запит із типовим заголовком `python-requests`	надіслати свій `User-Agent`; `osmnx` робить це сам
HTTP 429 або 504	сервер перевантажений або ви перевищили частоту	зачекати 30–60 с і повторити
`Connection refused`	за одним іменем стоїть кілька машин, і частина з них відмовляє	просто повторити: наступного разу випаде інша
запит висить і відвалюється	таймаут: запит завеликий	звузити район або теги

Про User-Agent є окрема пастка, на якій легко втратити півгодини. HTTP-заголовки передаються в кодуванні latin-1, тому кирилиця в рядку User-Agent кладе запит помилкою UnicodeEncodeError ще ДО відправлення — виглядає так, ніби винна мережа. Представляйтесь лише латиницею.

Скрипт `fetch_city_osm.py` перебирає дзеркала сам і бере перше, що відповідає на тестовий запит. Перелік чинних загальнодоступних дзеркал — на сторінці Overpass API. Якщо треба вказати своє, скористайтесь ключем `--overpass`.

Основні виклики `osmnx` — у довіднику бібліотеки:

```
ox.geocode_to_gdf('Львів, Україна')                  # межа міста
ox.features_from_polygon(poly, {'historic': True})   # об'єкти за тегами
ox.graph_from_polygon(poly, network_type='walk')     # мережа
```

2.4. Пішохідна мережа, а не автомобільна

`graph_from_polygon` приймає `network_type` зі значень `all`, `all_public`, `bike`, `drive`, `drive_service`, `walk`. Для цієї роботи потрібне **`walk`**, і причина не в правдоподібності сюжету.

У старому місті музеї стоять за сотню метрів один від одного, а **автомобільних** вузлів там одиниці: половина вулиць пішохідні, решта — з одностороннім рухом. Кілька пам'яток сідають на **один** вузол графа. Матриця віддалей після цього каже, що між двома різними музеями нуль метрів.

Це найнебезпечніший клас помилок у всьому курсі: програма не падає, повідомлення не з'являється, маршрут будується — і він неправильний. Єдиний захист — явна перевірка, яку ви напишете у кроці 4.

2.5. Нагадування про координати

Дані OSM приходять у **EPSG:4326** — градуси широти й довготи. Будь-яка довжина, площа чи буфер у градусах позбавлені сенсу: градус довготи у Львові коротший за градус широти приблизно в півтора раза.

Перед обчисленням метрів переходьте в метричну систему: для України це **EPSG:32635** (UTM зона 35N) або EPSG:6381 (УСК-2000).

```
gdf.to_crs(32635).area          # м², правильно
gdf.area                        # «квадратні градуси», безглуздо
```

> Документація geopandas про цю пастку не попереджає — сторінка про проєкції говорить лише про спотворення. Тому попереджаємо ми: projections.

Друга класична помилка — порядок координат. `shapely.Point` приймає **(x, y)**, тобто **(довгота, широта)**. У `folium`, навпаки, скрізь (lat, lon). Переплутаєте — об'єкт опиниться в іншій півкулі, і це видно одразу, якщо дивитись на карту.

3. Хід роботи

Одинадцять кроків. Після кожного є **контрольна ознака** — спосіб переконатись, що ви не поїхали в кювет. Контрольна ознака не є відповіддю: вона каже, на що дивитись, а не що там має бути.

Крок 0. Варіант

Впишіть прізвище та номер у списку групи. Місто підтягнеться зі списку автоматично. Якщо викладач призначив вам інше місто — перепишіть `CITY` вручну і зазначте це у звіті.

Крок 1. Дані двома способами

Допишіть функцію `fetch`: якщо файл шару вже збережено — читати з диска, не звертаючись до мережі; якщо ні — зробити живий запит, зберегти і записати, **скільки МБ пішло мережею і скільки секунд** це зайняло.

> Контрольна ознака: при ДРУГОМУ запуску ноутбука всі рядки таблиці вартості мають мати спосіб «знімок», а стовпець «передано, МБ» — нулі. Якщо ні — ваш `fetch` не кешує.

Крок 2. Хто претендує на звання пам'ятки

Відберіть кандидатів за об'єднанням `historic` і `tourism`, лишіть тільки тих, що мають `name`.

Безіменні об'єкти доводиться відкинути: маршрут не може містити точку «якась історична будівля». Це перше рішення, яке **відкидає частину даних**, і у звіті воно має бути назване разом із числом.

> Контрольна ознака: серед типів геометрії мають бути І `Point`, І `Polygon`. Самі лише полігони означають, що ви втратили пам'ятники, меморіали та оглядові точки.

Якщо кандидатів менше тридцяти. Невелике місто цілком може дати 15–20 об'єктів. Тоді або розширте перелік значень `tourism` (наприклад, додайте `theatre`), або зменште розмір великої задачі до того, що є. У звіті зазначте, що саме зробили і яке число отримали.

Крок 3. Критерій «найкращі»

Ключовий крок усієї роботи. Стовпця «цінність» у даних немає — ви його конструюєте.

Сигнали, доступні в OSM:

Сигнал	Що означає	Чому йому можна вірити
`wikidata` / `wikipedia`	про об'єкт є стаття	визнання ззовні OSM, незалежне джерело
кількість колонок `name:*`	назву переклали N мовами	перекладають те, що шукають іноземці
`tourism=museum`	це музей	призначений для відвідування
`historic` заповнений	має історичну природу	відсікає сучасні заклади
площа полігона	розмір будівлі	**найслабший**: великим буває і склад

Вимоги до вашої формули:

- використати **щонайменше три** різні сигнали;
- звести різнорідні величини до спільної шкали — інакше площа в тисячах квадратних метрів задавить двійковий прапорець (підказка: `rank(pct=True)` переводить будь-який стовпець у частки від 0 до 1);
- **пояснити у звіті, чому саме такі ваги**. Не «так вийшло», а міркування.

Обов'язкове порівняння

Побудуйте **другий** рейтинг за найпростішим критерієм — площею будівлі — і порахуйте, скільки об'єктів збігається в перших десяти.

> Обидва списки отримані з тих самих даних, тим самим кодом, без жодної помилки — і вони різні.
>
> Це не дефект аналізу. Це і є аналіз: питання «найкращі» не має відповіді в даних, відповідь задає той, хто обирає критерій. Саме це ви пояснюєте у звіті.

Дублікати

OSM наповнюють різні люди. Той самий музей трапляється двічі під трохи різними назвами; вежа буває окремим об'єктом усередині палацу, який теж у списку. Знайдіть у топ-30 пари, що стоять ближче **60 метрів** одна до одної, і вирішіть, що з ними робити.

> Контрольна ознака: якщо таких пар нуль — перевірте, чи працює ваша функція взагалі, на завідомо близьких точках. У місті з тридцятьма пам'ятками нуль близьких пар — рідкість.

Крок 4. Вокзал і прив'язка до графа

Пам'ятка — це будівля або вузол, а маршрут іде **ребрами графа**. Тому кожну точку доводиться «посадити» на найближчий вузол мережі. Операція втрачає точність, і втрату треба **виміряти**.

Напишіть `check_snap`, яка повідомляє: скільки точок і скільки **різних** вузлів вони зайняли; медіану й максимум відстані «точка → вузол»; а якщо вузлів менше, ніж точок — назви точок, що злиплись.

> Контрольна ознака: на пішохідній мережі різних вузлів має бути СТІЛЬКИ Ж, скільки точок. Менше — ви взяли автомобільну мережу, або місто надто мале, або два об'єкти справді є одним. Зафіксуйте у звіті: далі такі пари отримають нульову віддаль і маршрут обійде їх безплатно.

Крок 5. Матриця віддалей

Дві матриці: **пряма лінія** і **дорогою**. Їхнє відношення — коефіцієнт обходу — показує, наскільки пряма лінія бреше.

> Контрольні ознаки, звіряйтесь не підганяючи:
>
> • коефіцієнт обходу для пішохідної мережі у місті зазвичай у межах 1,2–1,6;
>
> • рівно 1,0 означає, що ви порівнюєте матрицю саму із собою;
>
> • більше 2 — або взято автомобільну мережу з односторонніми вулицями, або місто ділить річка чи залізниця, яку нема де перетнути. Друге — цікавий результат, перше — помилка. З'ясуйте, що саме у вас.

> Якщо серед віддалей трапились `inf`, частина графа не зв'язана з рештою. Не підміняйте `inf` великим числом мовчки — спершу подивіться, які це точки. Часто це об'єкт за містом або помилка в даних.

Зверніть увагу і на **асиметрію**: віддаль туди і назад може відрізнятись через односторонній рух. На пішохідній мережі вона мала, на автомобільній — помітна. Подивіться на своє число і поясніть його.

Кроки 6–7. Маршрут

Теорія — у розділі 4. Реалізувати треба два методи, третій дано готовим.

Крок 8. Індексація H3

Розкладіть **усіх** кандидатів по комірках H3 рівня 9 і побудуйте картограму щільності. Таблиця розмірів комірок: h3geo.org.

> Контрольна ознака: середнє ребро комірки рівня 9 має вийти близько 200 метрів. Якщо ваша карта опинилась в океані — ви переплутали порядок координат: `cell_to_boundary` повертає пари (lat, lng), а `shapely.Polygon` чекає (x, y), тобто (lng, lat).

Крок 9. Просторове поєднання

Для кожної точки маршруту знайдіть найближчий заклад із шару `amenities` і виведіть таблицю з відстанню в метрах. Потрібен `sjoin_nearest` з аргументом `distance_col` — документація.

> `sjoin_nearest` рахує відстань в одиницях CRS. У градусах результат буде безглуздим, і жодного попередження не буде — перепроєктуйте ОБИДВА шари в метричну систему до з'єднання.

Крок 10. Карта

Побудуйте карту, на якій **обидві** лінії: справжній маршрут по ребрах графа і прямі відрізки між точками. Різниця має бути видно оком.

> Контрольна ознака: якщо ваші дві лінії збігаються — ви намалювали прямі відрізки двічі. Маршрут по ребрах мусить огинати квартали.

Підкладка задається змінною `BASEMAP` на початку ноутбука. Підкладки CARTO з 2026 року вимагають ключ API і без нього повертають плитки з написом «API KEY REQUIRED» навскіс — тому взято `Esri.WorldGrayCanvas`, якому ключ не потрібен.

Крок 11. Результати для ЛР8

Збережіть `lr5_результати.json`. Лабораторна робота № 8 прочитає з нього матрицю віддалей великої задачі й спробує побити вашу довжину генетичним та мурашиним алгоритмами. Файл має бути відтворюваним: з тими самими даними ваш ноутбук має давати той самий результат.

4. Задача комівояжера

Дано множину точок і віддалі між ними. Знайти найкоротший замкнений маршрут, що проходить через кожну точку рівно один раз. У нашій постановці одна точка закріплена: маршрут починається і закінчується на вокзалі.

Задача належить до класу, для якого **не відомо** алгоритму, що розв'язує її швидко у загальному випадку. Це не означає, що вона безнадійна: для малих розмірів точний розв'язок цілком досяжний, а для великих існують методи, які дають добру відповідь без гарантії оптимальності.

4.1. Три методи

Метод	Що робить	Час	Що гарантує
найближчий сусід	щоразу йде в найближчу невідвідану точку	$O(n^2)$	нічого
2-opt	розвертає ділянки маршруту, поки довжина зменшується	$O(n^2)$ за прохід	локальний мінімум
Held–Karp	динамічне програмування за підмножинами	$O(2^n \cdot n^2)$	**точний оптимум**

Найближчий сусід

Стоїмо в поточній точці, дивимось на всі невідвідані, йдемо в найближчу. Повторюємо, поки не закінчаться, і повертаємось на старт.

Метод **жадібний**: він ухвалює рішення, яке виглядає найкраще зараз, і ніколи до нього не повертається. Типова плата — остання точка, до якої доводиться йти через усе місто, бо всі зручні сусіди вже витрачені.

2-opt

Бере готовий маршрут і пробує **розвернути** його ділянку: якщо маршрут іде A → B → … → C → D, спробувати A → C → … → B → D. Якщо стало коротше — лишаємо. Повторюємо, поки хоч одне розвертання допомагає.

Геометрично 2-opt прибирає **самоперетини**: маршрут, що перетинає сам себе, завжди можна скоротити розвертанням. Саме тому метод такий ефективний на географічних задачах.

Held–Karp

Динамічне програмування. Позначимо **g(S, j)** — довжину найкоротшого шляху, який починається на старті, проходить **рівно** через множину точок S і закінчується в точці j із S. Тоді

```
g(S, j) = min над i з S \ {j} :  g(S \ {j}, i) + d(i, j)
```

Відповідь — мінімум по j від `g(усі точки, j) + d(j, старт)`. Метод дає **гарантовано найкоротший** маршрут, і саме тому мала задача робиться на десяти точках: без точного розв'язку ви не дізнаєтесь, наскільки помиляються перші два методи.

4.2. Чому на тридцяти точках точного розв'язку немає

Таблиця **g** має $2^{n-1} \cdot (n-1)$ клітин. Порахуйте самі в ноутбуці — ось чого очікувати при 8 байтах на клітину:

Точок	Клітин у таблиці	Пам'яті	Операцій
11	$1{,}0 \cdot 10^4$	менше 1 МБ	$1{,}0 \cdot 10^5$
16	$4{,}9 \cdot 10^5$	4 МБ	$7{,}4 \cdot 10^6$
21	$2{,}1 \cdot 10^7$	0,17 ГБ	$4{,}2 \cdot 10^8$

Точок	Клітин у таблиці	Пам'яті	Операцій
26	$8{,}4 \cdot 10^{8}$	6,7 ГБ	$2{,}1 \cdot 10^{10}$
31	**$3{,}2 \cdot 10^{10}$**	**258 ГБ**	**$9{,}7 \cdot 10^{11}$**

> Зверніть увагу на природу перешкоди. Справа не в тому, що «довго» і треба почекати до ранку.

> Таблиця на 258 гігабайтів не вміщається в пам'ять жодного ноутбука. Точний розв'язок недоступний не через брак терпіння, а через брак пам'яті — і додавання ще п'яти точок множить потребу в тридцять разів.

> Відповідь «довго» у звіті не зараховується. Назвіть величину.

Залишаються наближені методи. Їхня спільна біда: **оптимум невідомий**, тож сказати «ми гірші на стільки-то відсотків» неможливо. Єдине, що лишається — порівнювати методи між собою.

4.3. Куди це веде

Ваш результат на великій задачі — довжина маршруту і час обчислення — стає **базою порівняння** для лабораторної роботи № 8. Там ту саму матрицю віддалей атакуватимуть генетичний і мурашиний алгоритми, і питання буде рівно одне: чи вдасться побити ваш 2-opt, і якою ціною в обчисленнях.

Тому `lr5_результати.json` має бути збережений акуратно: з матрицею віддалей, координатами точок і числами. Без нього ЛР8 не з чим порівнювати.

Докладніше про сам алгоритм: Held–Karp algorithm.

5. Типові помилки

Перелік зібраний із того, що справді ламається. Більшість із них **не викликають повідомлення про помилку** — програма працює, відповідь неправильна.

Симптом	Справжня причина	Як перевірити
дві пам'ятки за 0 метрів	злиплись на одному вузлі графа	`check_snap`: різних вузлів менше, ніж точок
маршрут «стрибає» через квартали	намальовані прямі відрізки замість геометрії ребер	подивитись на карту оком
`.area` дає мільйони	рахуєте у градусах	`gdf.crs` до обчислення
об'єкт опинився в іншій півкулі	переплутано (lat, lon) і (x, y)	карта; `Point` бере (довгота, широта)
маршрут не замкнувся	старт не доданий у кінець	`tour[0] == tour[-1] == 0`
`held_karp` зависає	запущено на 30 точках	таблиця зростання з розділу 4.2
у топ-10 два однакові музеї	дублікати в OSM не перевірено	пошук пар ближче 60 м
усі точки — полігони	шар узято з будівель, вузли втрачено	`geom_type.value_counts()`
`inf` у матриці	граф не зв'язаний	подивитись, які саме точки
плитки з написом «API KEY REQUIRED»	підкладка CARTO тепер вимагає ключ	`BASEMAP` на початку ноутбука

5.1. Про використання ШІ-помічників

Користуватись дозволено. **Неперевірено** користуватись — ні, і саме це оцінюється.

Мовні моделі впевнено пишуть геопросторовий код, який виглядає правильним і мовчки рахує не те. Три випадки, які трапляються постійно:

- обчислення площі чи віддалі без переходу в метричну систему координат;
- `Point(lat, lon)` замість `Point(lon, lat)`;
- виклики `osmnx` зі старого API: `geometries_from_place` прибрано у версії 2.0, лишилось `features_from_*`.

Тому у звіті зазначте, **де саме** ви користувались помічником і **як перевірили** результат. Відповідь «подивився, що працює» перевіркою не є: усі три помилки вище «працюють».

6. Вимоги до звіту

Звіт пишеться в останніх комірках ноутбука. Сім питань, на кожне — відповідь **зі своїми числами**.

№	Питання
1	Скільки об'єктів ви відкинули на кроці 2 і за яким правилом? Чи віддали б ви цей фільтр замовнику без пояснень?
2	Які сигнали ввійшли у ваш рейтинг і з якими вагами? Чому саме так?
3	Скільки об'єктів збіглося у ваших двох рейтингах із десяти? Що це означає для слова «найкращі»?
4	Який у вас коефіцієнт обходу і чи типовий він? Якщо ні — що в місті його пояснює?
5	На скільки відсотків найближчий сусід гірший за точний розв'язок? Чи здивувало вас це число?
6	Чому точний розв'язок на 30 точках недоступний? Назвіть конкретну величину.
7	Що у ваших даних виявилось неправильним — дублікати, пропущені назви, об'єкти не на місці? Як ви з цим вчинили?

> Питання 7 — найважливіше, і воно єдине, відповідь на яке не можна отримати з коду.

> Дані OpenStreetMap наповнюють волонтери. Там будуть дублікати, порожні назви і об'єкти, посаджені не на своє місце. Аналітик, який цього не помітив, здає замовнику красивий маршрут навколо неіснуючої пам'ятки.

Наприкінці звіту — **декларація про використання штучного інтелекту**.

7. Джерела

Що	Де
Довідник тегів OSM	wiki.openstreetmap.org/wiki/Map_Features
Ліцензія ODbL	openstreetmap.org/copyright
Мова запитів Overpass	Overpass QL
Дзеркала Overpass API	Overpass API
Довідник osmnx	osmnx.readthedocs.io
Просторове поєднання	geopandas: merging data
Системи координат	geopandas: projections
Розміри комірок H3	h3geo.org
Алгоритм Held–Karp	Held–Karp algorithm

Дані © OpenStreetMap contributors, ліцензія ODbL. Цей рядок має бути і у вашому звіті.